Otto-von-Guericke University Magdeburg



Interactive Product–Process–Resource (PPR) Modeling and Validation Tool

Submitted by:

Kishor Gurung (258441)
Abdul Rafay (258038)
Akash Anand Bhurke (259119)
Ayan Saikia (258961)

Submitted to:

Prof. Dr.- Ing. habil. Arndt Lüder

Fakultät für Maschinenbau
Otto-von-Guericke University
Universitätsplatz 2
39106 Magdeburg, Germany

Contents

Project Description

1 System Description
1.1 Application Description 1
1.2 Product-Process-Resource Representation 2
1.2.1 Valid Relationships 2
1.3 Supported Input Data Formats 3
1.3.1 Automation Markup Language (AML) 3
1.3.2 Microsoft Excel (XLSX) 5
1.4 Additional Views 7
1.4.1 Engineering View 7
1.4.2 Sustainability View 8
1.5 Requirement Checks 9
1.5.1 Requirement Checks on PPR Model 9
1.5.2 Requirement Checks on Additional Views 10

2 Software Architecture
2.1 Architectural Overview 11
2.2 Description of Utility Modules 15
2.2.1 AML Utilities (`aml_utils.py`) 15
2.2.2 Excel Utilities (`excel_utils.py`) 16
2.2.3 Graph Utilities (`graph_utils.py`) 17
2.2.4 Helper Functions (`helper_functions.py`) 18
2.2.5 PPR Conform Module (`ppr_conform.py`) 19
2.2.6 Requirement Check Module (`req_check.py`) 20
2.2.7 Viewpoints Menu Module (`viewpoints_menu.py`) 21
2.3 Description of Data Modules 22
2.3.1 Configuration Data (`config_data.py`) 22
2.3.2 Sample Data (`sample_data.py`) 23

3 Code Refactoring Principles
3.1 Modularization 24
3.2 Separation of Concerns 25
3.3 Extraction of Utility Functions 25
3.4 Removal of Duplicated Code 26

3.5 Improved Naming and Code Structure 27

4 Usability Principles

4.1 Clarity and Visual Differentiation 28
4.2 Mental Model Alignment 30
4.3 Simplicity and Minimalism 31
4.4 Consistency and Predictability 31
4.5 User Control and Freedom 32
4.6 Feedback and System Visibility 33
4.7 Error Handling and Prevention 34
4.8 Recognition over Recall 35
4.9 User Support 36

5 Demonstration Example

5.1 Import Model 38
5.1.1 File input 38
5.1.2 User Interface 39
5.1.3 PPR View 39
5.1.4 Engineering View 45
5.1.5 Sustainability View 48
5.2 Build PPR Model 49
5.3 Explore Sample 50
5.4 Help and Docs 50

List of Figures

1.1 Valid PPR Representation 2

2.1 High-Level Software Architecture of PPR Visualizer 12
2.2 Functional Logic of the Import Model Module 12
2.3 Interactive PPR View Architecture (Import/Build Mode) 13
2.4 Interactive Additional View Architecture (Import/Build Mode) . . . 13
2.5 Non-Interactive PPR View Architecture (Explore Sample Mode) . . . 14
2.6 Non-Interactive Additional View Architecture (Explore Sample Mode) 14

3.1 Extracted utility functions 26

4.1 Visualization of Node color, shape and size 29
4.2 Example of Edge label in Engineering view 29
4.3 Example of Edge label in Sustainability view 29
4.4 UI and its composition 31
4.5 Uniform placement of buttons and appropriate labelling 32
4.6 Feedback and system visibility 34
4.7 Error description of non-conform of PPR model in Model Import WS 35
4.8 Relevant dropdowns to reduce short-term memory demand 35
4.9 Help and Docs workspace in the app for user assistance 36

5.1 Initial User Interface 38
5.2 Steps for file input 38
5.3 Import Model - User Interface 39
5.4 PPR View - Steps for creating new node 39
5.5 PPR View - Steps for creating new edge 40
5.6 PPR View - Steps for updating node 41
5.7 PPR View - Updated node 41
5.8 PPR View - Steps for updating cardinality 42
5.9 PPR View - Updated cardinality 42
5.10 PPR View - Steps for deleting existing node 43
5.11 PPR View - Steps for deleting existing edge 43
5.12 PPR View - Steps for performing requirement check 44
5.13 Engineering View - Steps for adding node 45
5.14 Engineering View - Steps for creating new edge 46
5.15 Engineering View - Steps for deleting existing node 46

5.16 Engineering View - Steps for deleting existing edge 47
5.17 Engineering View - Steps for performing requirement check 47
5.18 Sustainability View - User Interface 48
5.19 Steps for switching from *Import Model* to *Build PPR Model* 49
5.20 Build PPR Model - User Interface 49
5.21 Explore Sample - User Interface 50
5.22 Help and Docs - User Interface 50

List of Tables

1 Project Task Distribution vi

1.1 Excerpt of the Node sheet 6
1.2 Excerpt of the Edges sheet 6

2.1 Functions in AML Utilities Module 16
2.2 Functions in Excel Utilities Module 17
2.3 Functions in Graph Utilities Module 18
2.4 Functions in Helper Functions Module 19
2.5 Functions in PPR Conform Module 20
2.6 Functions in Requirement Checks Module 21
2.7 Functions in Viewpoints Menu Module 21
2.8 Configuration Elements in Config Data Module 22
2.9 Sample Data in Sample Data Module 23

Project Description

The "Interactive Product-Process-Resource (PPR) Modeling and Validation Tool" is a final project developed within the **Python for Production Systems Engineering 2026** course. The primary objective is to design and implement a web-based engineering tool using **Streamlit library**, which serves as the core framework for building the interactive user interface and application logic. The tool facilitates the creation, modification, visualization, and validation of PPR models. The resulting application functions as a lightweight modeling workbench where users can import PPR models, manipulate their structures, and analyze them through graphs.

1 System Description

1.1 Application Description

The “Interactive Product–Process–Resource (PPR) Modeling and Validation Tool”, in this project, is named as the **Product–Process–Resource (PPR) Visualizer App.**

The application is designed as a **Streamlit-based** engineering tool that supports the creation, modification, visualization, and validation of Product–Process–Resource (PPR) models.
The main functionalities of the application are outlined below:

- The application supports building PPR models from scratch, as well as importing existing PPR models in a predefined format.
- It ensures that the PPR model conforms to a valid PPR structure through an integrated validation mechanism.
- It provides interactive graph-based visualization using yFiles for Streamlit, enabling intuitive exploration of the PPR structure.
- It supports multiple views, such as the Engineering view and the Sustainability view, on the same PPR model.
- It allows CRUD (Create, Read, Update, Delete) operations on the PPR model, as well as on additional views.
- It includes functionality to perform predefined requirement checks on the PPR model, as well as on any additional views created.

1.2 Product-Process-Resource Representation

A valid Product–Process–Resource (PPR) model is composed of three fundamental elements, each representing a distinct aspect of a production system:

- **Product**: Represents the item being created, modified, or transformed.
- **Process**: Denotes the value-adding activity that acts upon the product.
- **Resource**: Refers to the means or assets required to perform a process.

Each of these elements may further contain hierarchical child elements, enabling the representation of complex system structures.

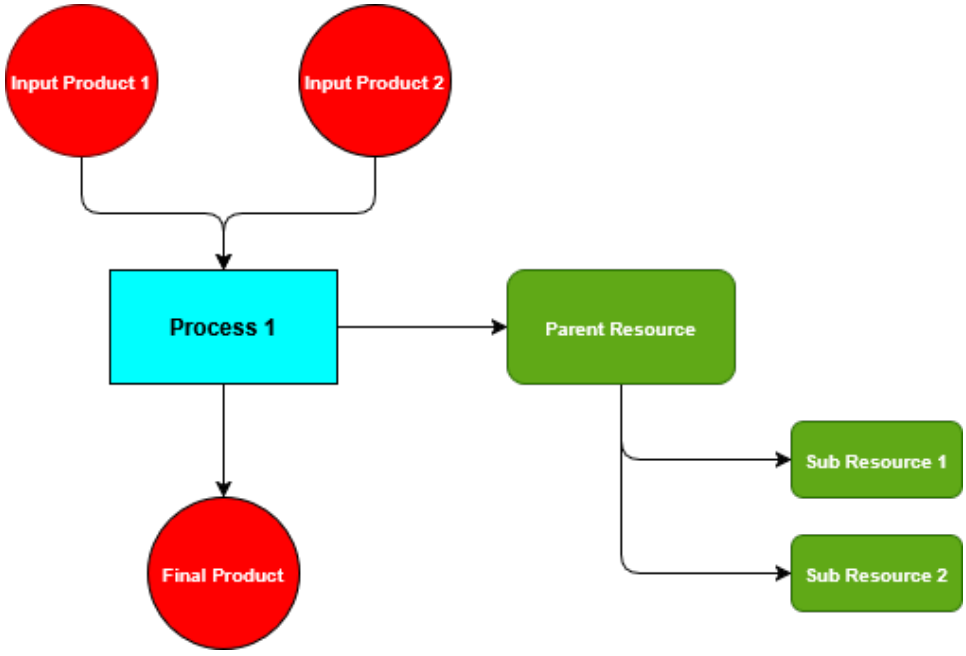


Figure 1.1: Valid PPR Representation

1.2.1 Valid Relationships

The validity of a PPR model is governed by a set of predefined connection rules between these elements. The following relationships are permitted:

- Product → Process
- Process → Product
- Process → Resource

In addition to the allowed relationships, the following constraints must be satisfied to ensure a valid PPR structure:

- A **Product instance** can be associated with only one **Process**.
- Direct connections between **Product** and **Resource** are not permitted.

1.3 Supported Input Data Formats

The application supports the following predefined input data formats:

- Automation Markup Language (AML)
- Microsoft Excel (XLSX)

1.3.1 Automation Markup Language (AML)

The AML format is used to define the structure of the Product–Process–Resource (PPR) model, including the class definitions and their associated attributes. The application requires a predefined AML schema in which the core PPR classes—*Product*, *Process*, and *Resource*—are specified.

Within this schema, attributes are defined at the class level. The *Product* class includes attributes such as `color` and `size`, whereas the *Process* and *Resource* classes do not contain any attributes.

In AML, direct element-to-element connections are not explicitly defined. Instead, relationships between elements are established using *External Interfaces*. Each instance of a PPR element contains two interfaces:

- `PPR_in` (for incoming connections)
- `PPR_out` (for outgoing connections)

Connections between elements are realized through links between these interfaces. In particular, the `PPR_out` interface of the *Product* class includes an additional attribute, `cardinality`, which specifies the number of product instances required for a given *Process*. This is necessary in scenarios where multiple instances of the same product are consumed by a single *Process*.

A simplified excerpt of the AML class and interface definition is shown below:

```
<InterfaceClassLib Name="InterfaceClassLib">
  <InterfaceClass Name="PPR_interface">
    <ExternalInterface Name="PPR_in" />
    <ExternalInterface Name="PPR_out" />
  </InterfaceClass>
</InterfaceClassLib>

<SystemUnitClassLib Name="SystemUnitClassLib">
  <SystemUnitClass Name="Product">
    <Attribute Name="color" />
    <Attribute Name="size" />
    <ExternalInterface Name="PPR_in" />
    <ExternalInterface Name="PPR_out">
      <Attribute Name="cardinality" />
    </ExternalInterface>
  </SystemUnitClass>

  <SystemUnitClass Name="Process">
    <ExternalInterface Name="PPR_in" />
    <ExternalInterface Name="PPR_out" />
  </SystemUnitClass>

  <SystemUnitClass Name="Resource">
    <ExternalInterface Name="PPR_in" />
    <ExternalInterface Name="PPR_out" />
  </SystemUnitClass>
</SystemUnitClassLib>
```

1.3.2 Microsoft Excel (XLSX)

The Microsoft Excel (XLSX) format provides a tabular representation of the Product–Process–Resource (PPR) model. It serves as a simplified and user-friendly alternative to AML for defining PPR elements and their relationships.

The Excel file is structured into two worksheets:

- **Nodes**: defines all PPR elements and their attributes
- **Edges**: defines the relationships between these elements

Nodes

The *Nodes* sheet contains a list of all PPR elements. Each row represents a single node in the PPR model. The following columns are required:

- `Node_ID`: A unique identifier for each element
- `Name`: A descriptive name of the element
- `PPR_Type`: Specifies the type of the element (*Product*, *Process*, or *Resource*)
- `Color` (optional): Attribute applicable only to *Product* elements
- `Size` (optional): Attribute applicable only to *Product* elements

Attributes such as `Color` and `Size` are only relevant for *Product* nodes. For *Process* and *Resource* nodes, these fields remain empty.

Edges

The *Edges* sheet defines the relationships between nodes. Each row represents a directed connection from one node to another. The following columns are required:

- `Start Node`: The identifier of the source node
- `End Node`: The identifier of the target node

- `Cardinality` (optional): Specifies the quantity of *Product* instances required

The `Cardinality` attribute is only applicable to edges originating from *Product* nodes and defines how many instances of that Product are required by the target *Process*. If no value is specified, a default cardinality of one is assumed.

A simplified excerpt for the two sheets in XLSX format is shown below:

Node_ID	**Name**	**PPR_Type**	**Color**	**Size**
P1	Plastic	Product		
P5	Tyre	Product	black	small
P7	Wheel	Product	black	medium
PR2	Mating	Process		
PR3	Wheel Mounting	Process		
R5	Mating Robot	Resource		

Table 1.1: Excerpt of the Node sheet

Start Node	**End Node**	**Cardinality**
P5	PR2	
P6	PR2	
P7	PR3	4
P8	PR3	2
PR2	P7	
PR3	P9	

Table 1.2: Excerpt of the Edges sheet

The input data (PPR data), regardless of whether it is provided in AML or Excel format, is parsed into a unified internal representation within the application. The elements of the PPR model are stored as a *Python dictionary of nodes*, where each node represents an individual Product, Process, or Resource along with its associated attributes.

The relationships between these elements are represented as a *Python list of edges*, where each edge defines a connection between two nodes and may include additional properties such as cardinality. Both the node dictionary and edge list are stored in the Streamlit session state, ensuring consistent data availability across different stages of the application workflow.

1.4 Additional Views

The application supports the creation, modification, visualization, and validation of additional views based on the same underlying PPR model. These views enable domain-specific analysis without altering the core model structure. The following views are supported in the application:

- Engineering View
- Sustainability View

1.4.1 Engineering View

The Engineering View represents the core engineering description of the production system. It allows the assignment of predefined attributes and relationships to PPR nodes in order to capture performance-related and operational characteristics.

Attributes

Each PPR node type is associated with specific engineering attributes:

- **Product**:
  - `product_cost` (float, Euro): Represents the cost associated with the product.
- **Process**:
  - `cycle_time` (float, seconds): Defines the time required to complete the process.
- **Resource**:
  - `operating_speed` (float, Units/min): Indicates the operational speed of the resource.
  - `operating_cost` (float, Euro): Represents the cost of operating the resource.

Edges

The Engineering View supports the following edge types:

- `part_of`
- `contributes_to`
- `impacts`
- `constraints`

These edges enable a more detailed representation of engineering dependencies and interactions within the production system.

1.4.2 Sustainability View

The Sustainability View represents the environmental perspective of the production system. It allows the assignment of predefined attributes and relationships to PPR nodes in order to capture environmental and sustainability-related characteristics.

Attributes

Each PPR node type is associated with specific sustainability attributes:

- **Product**:
  - `durability` (int, years): Represents the expected lifespan of the product.
  - `co2_per_unit` (float, kg/unit): Indicates the carbon dioxide emissions associated with each unit of the product.
- **Process**:
  - `specific_energy_consumption` (float, kW/unit): Defines the energy required per unit during the process.
  - `co2_emission` (float, kg/hr): Represents the rate of carbon dioxide emissions generated by the process.
- **Resource**:
  - `energy_flow_rate` (float, kW/hr): Indicates the rate of energy flow associated with the resource.

Edges
The Sustainability View supports the following edges types:

- `energy_flows`
- `material_flows`
- `influences`

These edges enable a more detailed representation of environmental interactions and sustainability-related dependencies within the production system.

1.5 Requirement Checks

The application performs requirement checks on the PPR model and on the additional views created. These checks help verify the structure of the model and support basic analysis based on node attributes.

1.5.1 Requirement Checks on PPR Model

The requirement checks on the PPR model focus on validating the structural integrity and logical consistency of the model.

- The Requirement check groups PPR nodes according to their types, namely *Product*, *Process*, and *Resource*, and visualizes the model accordingly to improve interpretability.
- For a specific use case representing the production of a Lego car, the application verifies whether an *Axle* is connected to exactly two *Wheel* components through a *Process*. This ensures that domain-specific production constraints are satisfied.
- The requirement check allows identification of isolated nodes within the PPR model. This includes detecting incomplete or disconnected production flows where nodes are not properly linked to the overall system.

1.5.2 Requirement Checks on Additional Views

For additional views, the requirement checks focus on evaluating node attributes and enabling analytical queries based on defined constraints.

- The requirement check allows validation checks on the attributes assigned to PPR nodes within the additional views, ensuring that they satisfy certain conditions and constraints.

2
Software Architecture

2.1 Architectural Overview

The **Product–Process–Resource (PPR) Visualizer** mainly utilizes the following Python libraries.

- **Streamlit**: Used as the primary framework for building the interactive web-based user interface.
- **yFiles for Streamlit**: Used for interactive graph visualization of the PPR model.
- **Pandas**: Used for structured data handling.
- **XML ElementTree**: Used for parsing and processing AML (CAEX) files.
- **NetworkX**: Used internally for graph construction.

Using the above libraries, a multi-layered and modular software architecture of the app is designed.
The following diagrams offer a decomposition of the software architecture, beginning with a high-level overview and progressing into the specific logic of individual modules. These components are presented within the context of the system's two primary operational workflows: *Import/Build Model* and *Explore Sample*.

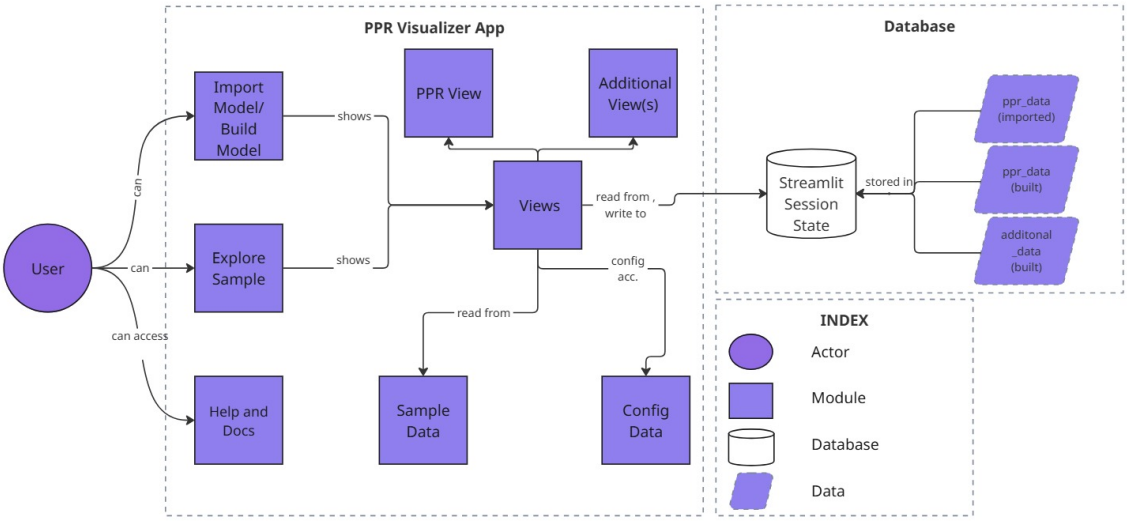


Figure 2.1: High-Level Software Architecture of PPR Visualizer

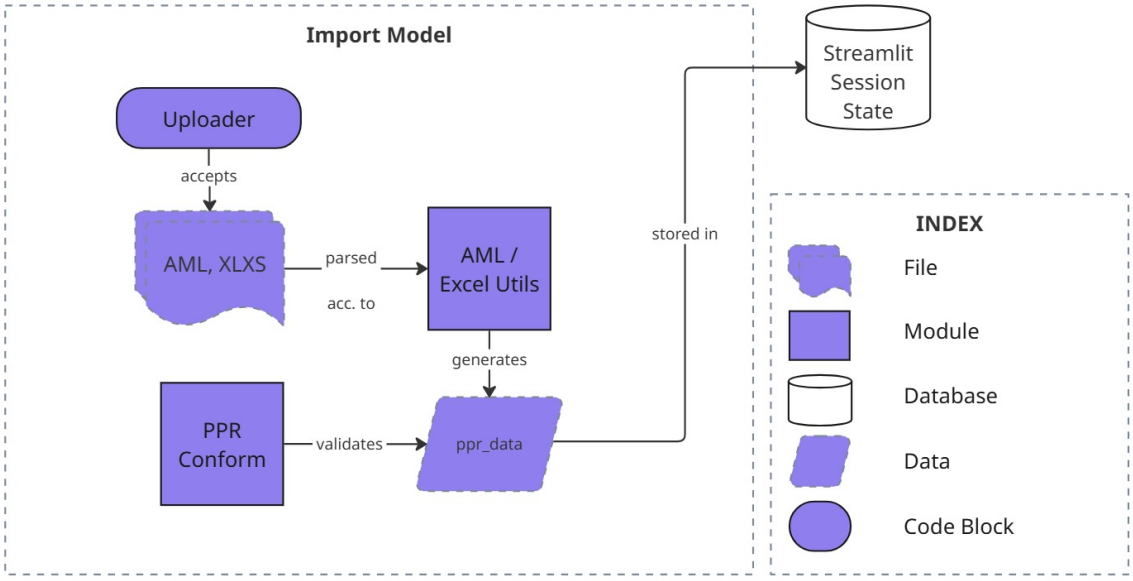


Figure 2.2: Functional Logic of the Import Model Module

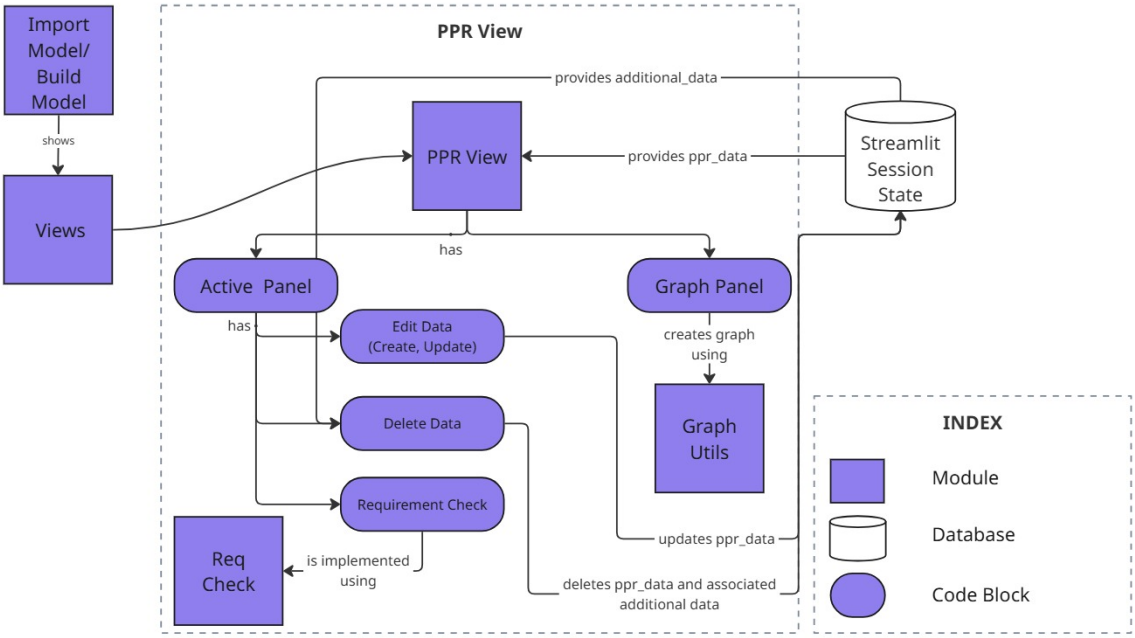


Figure 2.3: Interactive PPR View Architecture (Import/Build Mode)

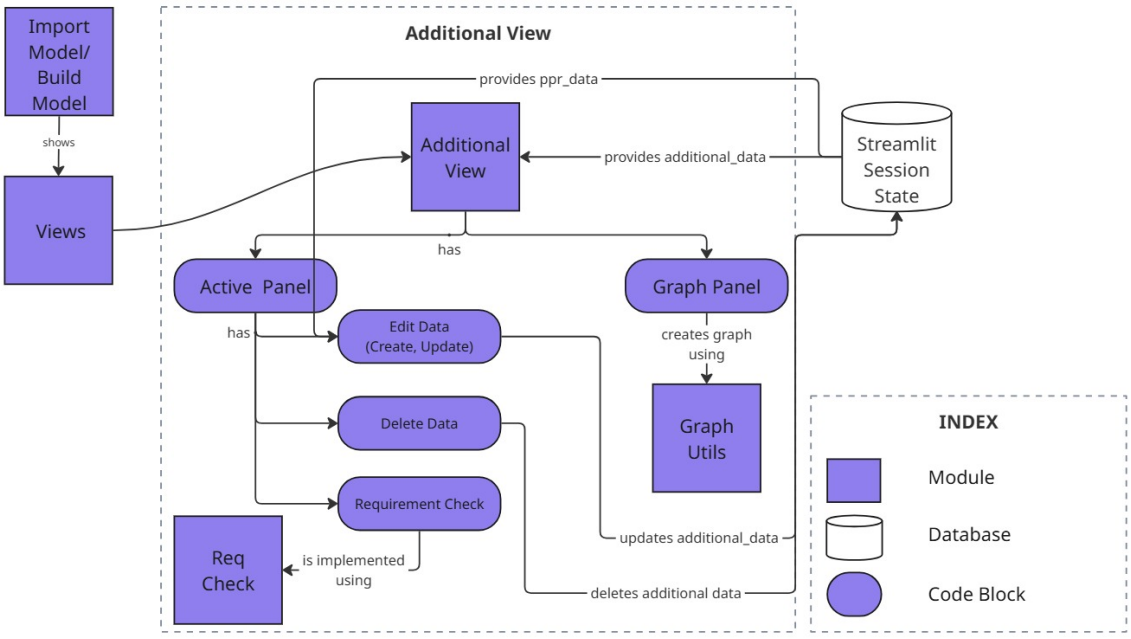


Figure 2.4: Interactive Additional View Architecture (Import/Build Mode)

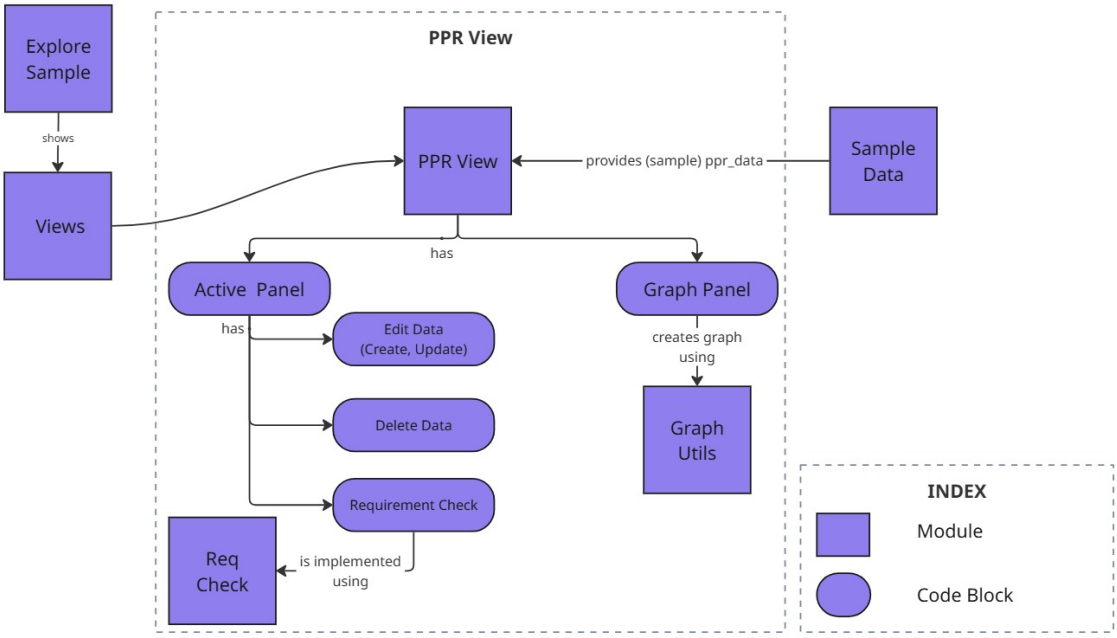


Figure 2.5: Non-Interactive PPR View Architecture (Explore Sample Mode)

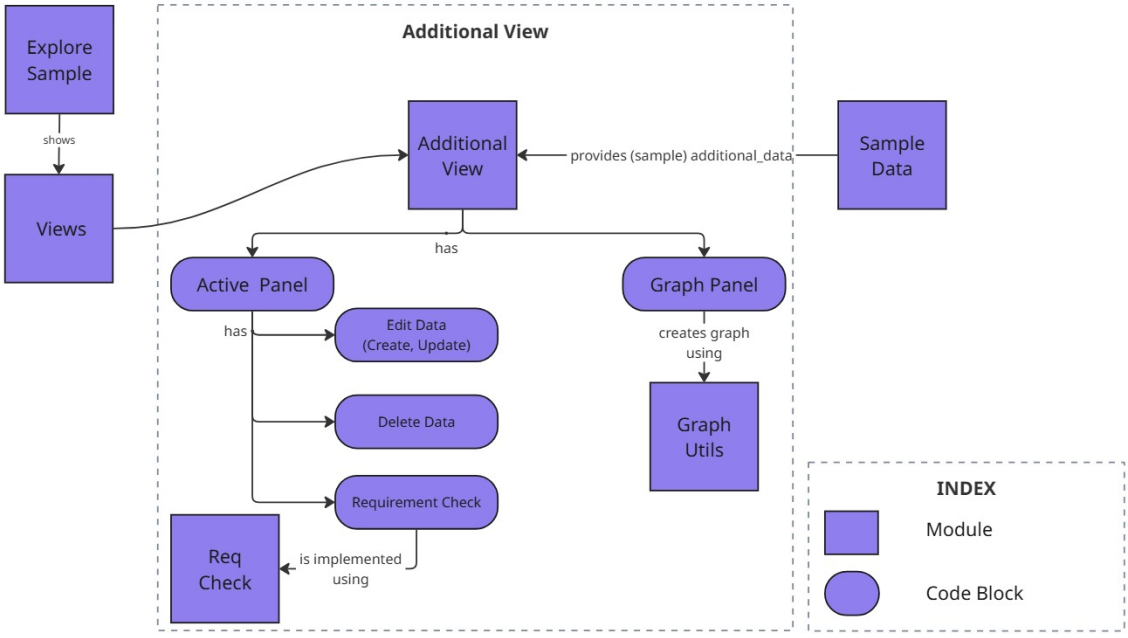


Figure 2.6: Non-Interactive Additional View Architecture (Explore Sample Mode)

2.2 Description of Utility Modules

The PPR Visualizer App utilizes a set of utility modules that encapsulate reusable functionalities required across different components of the application. This separation improves modularity, reduces code duplication, and enhances the overall maintainability of the system.

The following utility modules are used within the application:

- AML Utilities (`aml_utils.py`)
- Excel Utilities (`excel_utils.py`)
- Graph Utilities (`graph_utils.py`)
- Helper Functions (`helper_functions.py`)
- PPR Conformance Module (`ppr_conform.py`)
- Requirement Check Module (`req_check.py`)
- Viewpoints Menu Module (`viewpoints_menu.py`)

2.2.1 AML Utilities (`aml_utils.py`)

This module handles the processing of AML (Automation Markup Language) file and converts it into PPR data.

The functions defined in this module are listed in Table 2.1.

Function	Description
`build_elements_dict`	Extracts PPR elements from the AML file and organizes them into a structured dictionary of PPR nodes, including hierarchical relationships.
`extract_links`	Retrieves raw interface-based connections from the AML file and oragnizes them into a structured list of PPR edges (interface-based links) without mapping them to specific PPR nodes.
`iface_mapping`	Maps external interfaces to their corresponding PPR nodes, enabling interpretation of interface-level connections.
`extract_ppr_links`	Converts interface-based links (edges) into node-level relationships between Product, Process, and Resource.

Table 2.1: Functions in AML Utilities Module

2.2.2 Excel Utilities (`excel_utils.py`)

This module handles the processing of XLSX file and converts it into PPR data.

The functions defined in this module are listed in Table 2.2

Function	Description
`validate_excel`	Validates the structure of the Excel file. Ensures required sheets (*Nodes*, *Edges*) and mandatory columns exist, and raises errors if the format is invalid.
`build_node_lookup`	Creates a lookup dictionary mapping `Node_ID` to node metadata (name and PPR type) for fast access during edge processing.
`validate_cardinality`	Validates and converts the `Cardinality` field in the Edges sheet, ensuring it is either an integer or empty.
`elements_from_excel`	Extracts PPR nodes from the Nodes sheet and converts them into structured element dictionaries, including attributes such as color and size.
`links_from_excel`	Extracts relationships from the Edges sheet and maps them to PPR node-level links, including cardinality and parent–child classification.
`apply_parent_relationships`	Assigns hierarchical parent relationships between elements based on detected parent–child links.
`parse_excel`	Main entry function that orchestrates validation, node extraction, link extraction, and relationship assignment to produce the final PPR model.

Table 2.2: Functions in Excel Utilities Module

2.2.3 Graph Utilities (`graph_utils.py`)

This module handles graph-related operations in the application. It is used to construct the PPR graph, compute basic properties, and prepare the graph for visualization.

The functions defined in this module are listed in Table 2.3.

Function	Description
`nx_DiGraph`	Builds a directed graph from the PPR data using the NetworkX library.
`node_cardinality_mapping`	Computes edge-related properties such as cardinality between nodes.
`class_to_shape`	Maps PPR node types (Product, Process, Resource) to visual styles for graph representation.
`nx_to_yfiles_graph`	Converts the NetworkX graph into a format compatible with yFiles for visualization.
`show_yfiles_graph`	Renders the interactive graph in the Streamlit interface.

Table 2.3: Functions in Graph Utilities Module

2.2.4 Helper Functions (`helper_functions.py`)

This module contains light-weight utility functions used throughout the application for processing and managing PPR model data structures, specifically nodes (dictionary-based) and edges (list-based).

The functions defined in this module are listed in Table 2.4.

Function	Description
`nodes_by_class`	Groups nodes based on their class (e.g., Product, Process, Resource) and returns a dictionary mapping each class to a list of node names.
`edges_by_type`	Groups edges based on their type and returns a dictionary mapping each type to a list of edge names.
`extract_numeric`	Extracts the numeric value from a string containing a number and unit (e.g., "12.5 kg"). Returns 0.0 if extraction fails.
`get_cardinality_by_name`	Retrieves the cardinality value of a link by its name from a list of links, returning a default value if not found.
`are_attributes_same`	Compares existing node attributes with a new set of selected attributes.
`remove_links_of_node`	Removes all links from a list where the specified node appears as either source or target.
`has_descendants`	Checks whether a given node has child nodes (i.e., descendants) in the node dictionary.
`delete_node_recursive`	Recursively deletes a node and all its descendants, removing associated links and entries from multiple node and edge collections.

Table 2.4: Functions in Helper Functions Module

2.2.5 PPR Conform Module (`ppr_conform.py`)

This module contains functions which allow to validate whether imported data conforms to PPR (Product–Process–Resource) structure. It is used after importing AML/XLSX data and before visualization to ensure that the graph structure conforms to valid PPR representation. All validation functions return a list of errors describing violations, which can be displayed in the Streamlit interface

The validation functions defined in this module are listed in Table 2.5.

Function	Description
`validate_one_product_one_process`	Ensures that each Product node is connected to only one Process node, enforcing a one-to-one relationship.
`validate_product_to_resource`	Checks for invalid direct connections between Product and Resource nodes, which violate PPR semantics.
`validate_cardinality`	Verifies that cardinality is defined only for Product–Process relationships and that the values are numeric.
`validate_nodes`	Ensures that all nodes have valid classifications (Product, Process, Resource) and that no node has missing or invalid class information.
`validate_edges`	Aggregates and executes all edge-related validations, including product-process constraints, invalid connections, and cardinality checks.

Table 2.5: Functions in PPR Conform Module

2.2.6 Requirement Check Module (`req_check.py`)

This module contains functions that perform requirement validations on the PPR model, as well as on any addiional views.

The functions defined in this module are listed in Table 2.6.

Function	Description
`ppr_req_checks`	Provides interactive requirement checks using the Streamlit UI. Includes validation of Axle–Wheel connectivity (ensuring correct cardinality relationships) and detection of disconnected (isolated) nodes in the graph.
`get_matching_elements_by_attribute`	Filters nodes based on attribute conditions for additional viewpoints (e.g., engineering or sustainability). Supports comparison operations (=, <, >, <=, >=) and returns matching elements as a pandas DataFrame.

Table 2.6: Functions in Requirement Checks Module

.

2.2.7 Viewpoints Menu Module (`viewpoints_menu.py`)

This module manages viewpoint selection in the Streamlit interface. It allows users to switch between different views (e.g., PPR, Engineering, Sustainability) while maintaining the selected state using Streamlit session state.

The functions defined in this module are listed in Table 2.7.

Function	Description
`handle`	Displays a horizontal viewpoint menu and updates the selected view using `st.session_state`, ensuring persistence across user interactions and sidebar contexts.

Table 2.7: Functions in Viewpoints Menu Module

2.3 Description of Data Modules

The PPR Visualizer App utilizes a set of data modules to manage input data and configuration settings required for the operation of the application. These modules provide structured data that supports model creation, validation, and analysis.

The following data modules are used within the application:

- Configuration Data (`config_data.py`)
- Sample Data (`sample_data.py`)

2.3.1 Configuration Data (`config_data.py`)

This module defines configuration settings and constants used throughout the application. The configuration elements defined in this module are listed in Table 2.8.

Element	Description
`global_nodes_list`	Defines the valid PPR node classes (Product, Process, Resource) used for validation and classification.
`ppr_edge_type_mapping`	Specifies allowed relationships between node types in the PPR model (e.g., Product → Process).
`edgeType_by_view`	Maps each viewpoint (PPR, Engineering, Sustainability) to its corresponding edge types.
`PPR_ATTRS`	Defines attribute structures for nodes in the PPR view, including categorical options.
`ENG_ATTRS`	Defines attribute schemas for the Engineering view, including numeric values with units.
`SUST_ATTRS`	Defines attribute schemas for the Sustainability view, including environmental metrics.
`VIEW_ATTRS_MAPPING`	Links each viewpoint to its corresponding attribute definitions, enabling dynamic attribute handling.

Table 2.8: Configuration Elements in Config Data Module

2.3.2 Sample Data (`sample_data.py`)

This module provides pre-defined sample datasets for Explore Sample module.

The datasets defined in this module are listed in Table 2.9.

Element	Description
`sample_ppr_nodes`	Contains sample PPR nodes with attributes, class types, hierarchy, and identifiers.
`sample_ppr_edges`	Contains sample PPR relationships between nodes, including edge types and cardinalities.
`sample_eng_nodes`	Contains sample nodes with Engineering-specific attributes (e.g., cost, cycle time).
`sample_eng_edges`	Contains Engineering view relationships such as `part_of`, `contributes_to`, and `constraints`.
`sample_sus_nodes`	Contains sample nodes with Sustainability attributes (e.g., energy consumption, CO_2 metrics).
`sample_sus_edges`	Contains Sustainability view relationships such as `energy_flows`, `material_flows`, and `influences`.

Table 2.9: Sample Data in Sample Data Module

3

Code Refactoring Principles

During the development of the PPR visualization application, multiple software engineering refactoring principles were applied to improve maintainability, scalability and readability of the codebase.

3.1 Modularization

The application was structured into separate logical modules for both UI and background processing rather than a monolithic script. Key separations include:

- Data configuration
- UI components (Streamlit controls, sidebar)
- Utility functions
- View specific workflow
- Graph visualization logic (yFiles rendering)
- Data parsing (AML/XLSX processing)
- CRUD opration logic
- Requirement check logic

This modularization ensures that each component can be developed and tested independently. It helps in bug identification and faster resolution.

For example, the PPR viewpoint and task specific viewpoints graphs are handled as separate modules (eg. `ppr_view.py` and

`additional_views.py`), enabling independent updates of viewpoint configurations without affecting the entire system.

3.2 Separation of Concerns

Each module is responsible for a single concern: The internal modules are separated to provide clear output from specific requirement.

- Data layer: Handles AML/XLSX parsing and transformation into node-edge structures
- Independent workflow with session states for Creating Model, Importing model and Example exploration
- View specific workflow
- PPR conformity check: Applies requirement validation rules
- Presentation layer: Streamlit UI + graph rendering

This separation reduces coupling and improves extensibility. For instance, changes in AML/XLSX schema parsing do not affect UI logic.

3.3 Extraction of Utility Functions

Reusable logic was abstracted into utility/helper functions such as:

- Viewpoints Menu
- Validation of one Product $\rightarrow$ one process relation
- Checking Cardinality and Node-Edge Validation
- Node and edge processing helper functions
- AML and XLSX parsing helpers
- Graph transformation utilities
- View Specific Requirement validation functions

This avoids repeated code blocks and improves readability. For example:

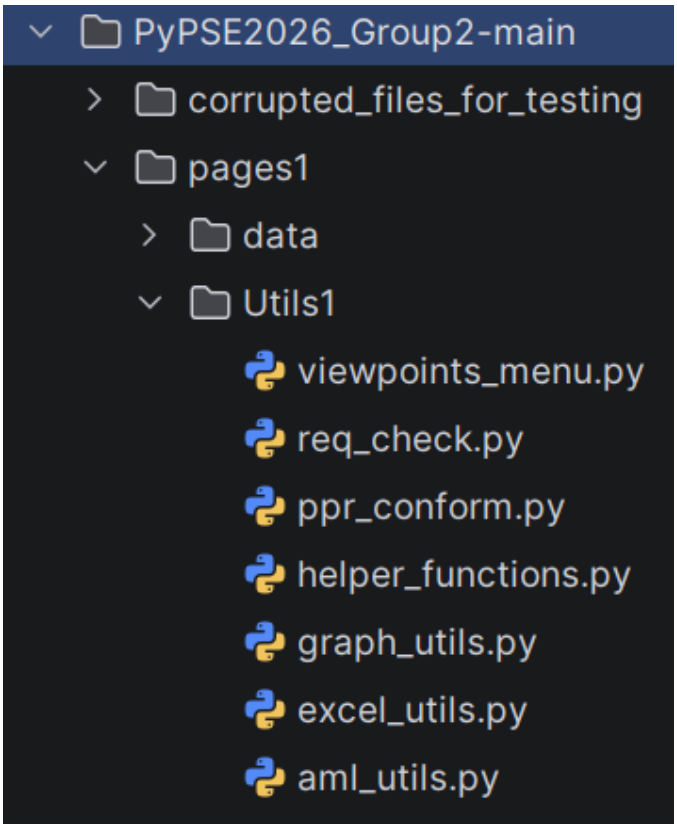


Figure 3.1: Extracted utility functions

- Instead of repeating viewpoint menu function in each view, a single function is introduced.
- Node styling or classification logic is standardized and centralized.
- Central data processing utilities are introduced to support editing, validation of the PPR data.

3.4 Removal of Duplicated Code

Duplicate logic especially in:

- Viewpoint generation
- UI and processsing logic for Model Import, Model Creation and Sample Exploration

- Graph rendering
- Sidebar interactions

was eliminated by introducing reusable functions and configuration-driven approaches.

For example:

- Instead of duplicating graph rendering logic for different views, a parameterized graph builder function is used.
- Single source generation of viewpoint based on requirement is introduced.
- UI for specific operations is Centralised.

3.5 Improved Naming and Code Structure

Clear and domain-specific naming conventions were adopted to make its purpose clearer. For example:

- Variables reflect domain meaning (e.g., `product_nodes`, `process_nodes`, `parent_node` etc.)
- Functions are action-oriented (e.g., `extract_ppr_links()`, `validate_nodes()`, `node_cardinality_mapping()` etc.)

This aligns the code with PPR domain terminology (Product–Process–Resource), improving readability for engineering stakeholders.

4

Usability Principles

The application interface was designed using established usability principles such as those from Stanford Improvement Analytics and Innovation Services, which emphasize clarity, simplicity, feedback, and consistency in user interfaces.

An extensive Product–Process–Resource (PPR) graph visualiser is incorporated. This can illustrate how the product flows through the system, which processes are being carried out and which resources are required for particular process.

4.1 Clarity and Visual Differentiation

The structure ensures that different node types (Product, Process, Resource) are visually distinguishable through:

- Different shapes for nodes(via yFiles configuration)
- Node Color coding and size depending on parent-child relation
- Node Labels and grouping with respect to class
- Edge labels clearly defined for addtional viewpoints

Workflow is structured in 3 separate continuous workflows:

- Viewpoint Selection (PPR, Basic Engineering, Sustainability)
- Panel options (Modify, Delete, Requirement Check)
- Specific Operation (Create, Edit, Delete, View specific requirement check)

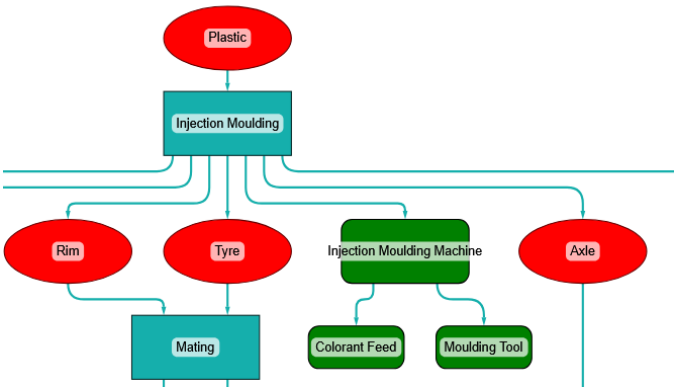


Figure 4.1: Visualization of Node color, shape and size

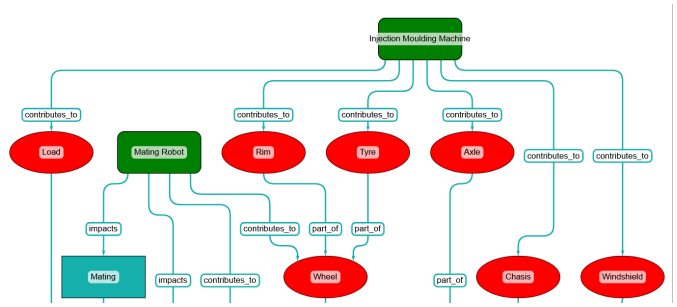


Figure 4.2: Example of Edge label in Engineering view

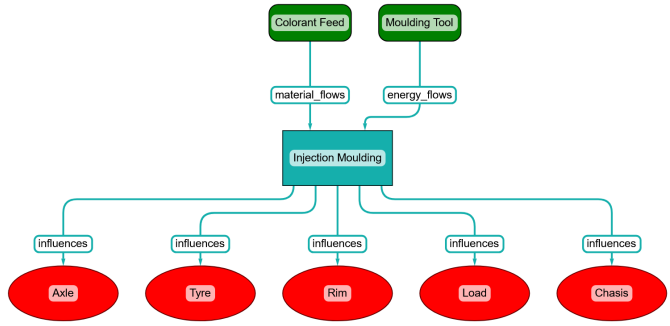


Figure 4.3: Example of Edge label in Sustainability view

4.2 Mental Model Alignment

The application follows the real-world PPR structure used widely in the industry, similar to PPR graphs of Enterprise Architect, Capella etc. matching how users conceptualize manufacturing systems.

- Products → Processes → Resources

For edge creation, practical usage scenarios were taken into account by incorporating exception handling and enforcing constraints that ensure only valid connections can be established. These rules are designed to maintain strict adherence to PPR conformity while still allowing necessary flexibility. The following rules have been defined:

- Three types of edges are defined:
  - Product → Process
  - Process → Resource
  - Process → Product
- A product node can be a source node for only one process.

If a user attempts to create an invalid edge, a clear PPR conformity violation message is displayed to inform them of the issue. This aligns with the mental models that the user holds of the PPR system, where systems reflects real-world understanding.

4.3 Simplicity and Minimalism

The UI avoids unnecessary complexity for the user by introducing:

- An intuitive and streamlined sidebar for navigation
- Dedicated visualizations tailored to each specific view
- A clearly separated graph visualization area distinct from the editing interface
- A concise set of essential controls, including edit, delete, and requirement check

This follows the principle of simplicity, ensuring users focus only on essential tasks.

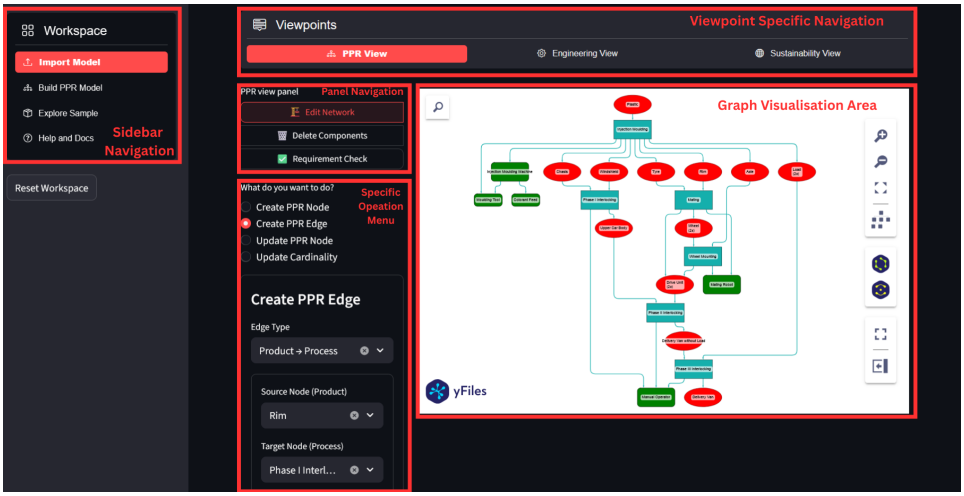


Figure 4.4: UI and its composition

4.4 Consistency and Predictability

Consistent UI patterns are maintained across the different menus. For example:

- The interface uses the same name format for menu items as their corresponding viewpoint selection.

- The Main menu is always in the sidebar, viewpoint selection is always on top and the graph visualization is always on the right and does not change on different workspaces of the interface.
- Same interaction logic across views (PPR, Engineering & Sustainability)
- Uniform button styles, labels and placement corresponding to the action taken.
- Predictable graph interaction behavior (zoom, select, edit) using yFiles configuration.

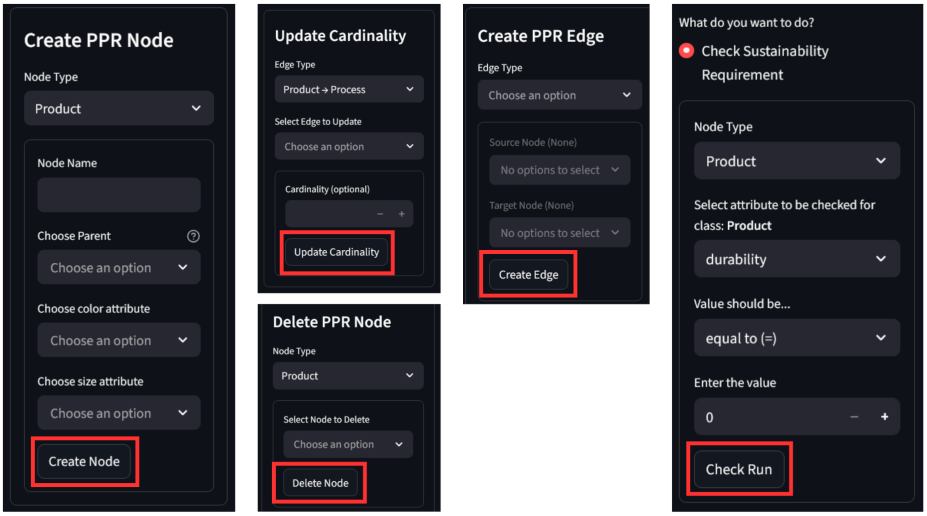


Figure 4.5: Uniform placement of buttons and appropriate labelling

This reduces cognitive load, creates consistency and makes the app predictable for user.

4.5 User Control and Freedom

The interface is designed to provide users with a clear sense of control, enabling them to interact with the system confidently and effectively. It also ensures that users always remain aware of their current position within the application.

- Users are given full control over Workspace navigation, viewpoint navigation, Operations panel and individual operation- allowing them to proceed to the required workspace at their own discretion rather than being redirected automatically.
- Tasks are designed for efficient completion, minimizing the need for workarounds or repetitive steps. For instance, the graph dynamically updates in response to CRUD operations, removing the need for any manual refresh or intervention.

4.6 Feedback and System Visibility

The interface continuously keeps users informed about system status by providing timely and relevant feedback, ensuring they are always aware of ongoing processes and outcomes within a reasonable response time.

- Confirmation message upon action completed are shown, confirming to the user that it was a success.
- Requirement Check messages are clearly shown, with relevant information added to it based on the user input.
- Visual highlights in Red for errors or violations, Green for successful operation and Yellow for additional information.

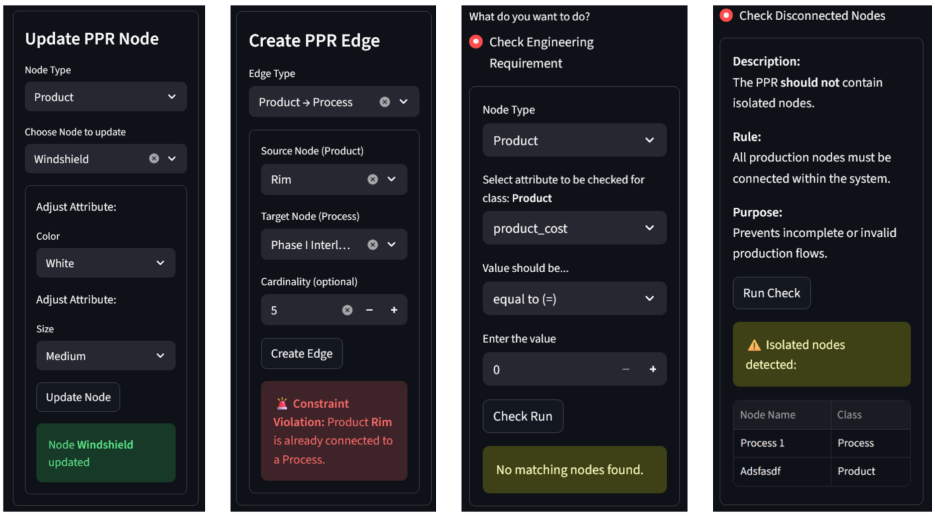


Figure 4.6: Feedback and system visibility

This ensures users are always aware of system state, clear feedback to help user correct relevant mistakes rapidly and builds trusts of consistent behaviour.

4.7 Error Handling and Prevention

The system is designed to minimize the occurrence of errors and, when they do arise, to present clear and meaningful messages that enable straightforward handling.

- In case of Model Import, App checks for correct format both in AML and XLSX and provides feedback and prevent incorrect data entry.
- Form fields are validated to reduce the likelihood of user errors and ensure accurate data input.
- Clear error messages in simple language are included to guide to corrective action.

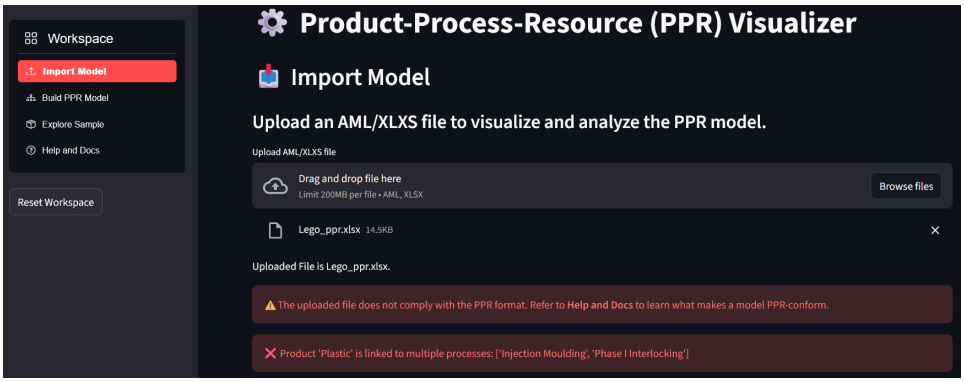


Figure 4.7: Error description of non-conform of PPR model in Model Import WS

4.8 Recognition over Recall

The app minimizes short-term memory demands on users by presenting relevant information and contextual assistance directly within the interface.

- Relevant Dropdowns, labels, and structured UI reduce need to remember inputs
- Dynamic Visual graph with attribute visualisation by hovering reduces cognitive effort.

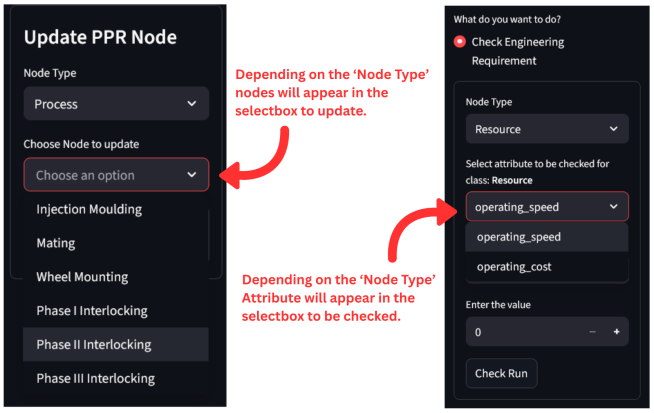


Figure 4.8: Relevant dropdowns to reduce short-term memory demand

4.9 User Support

The interface provides additional assistance for PPR conformity, complete documentation as needed or requested.

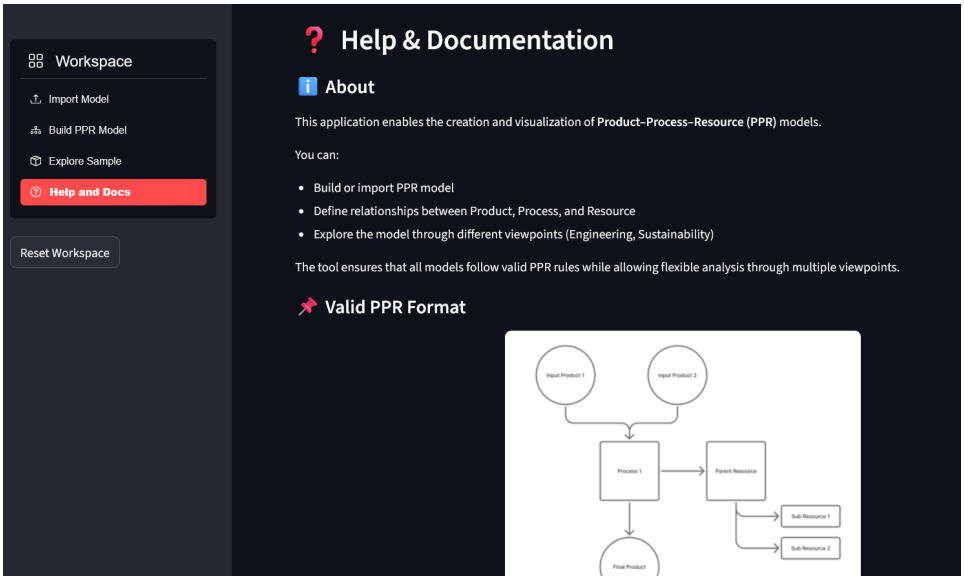


Figure 4.9: Help and Docs workspace in the app for user assistance

5

Demonstration Example

This section of the documentation demonstrates how to view and work with an existing input file in AML or XLSX format with our PPR Modeling and Validation Tool.

It is a step-by-step walkthrough that illustrates the main features of the tool, including file input and graph visualization, alongside creating, deleting, and updating of nodes and edges in different viewpoints, as well as performing requirement checks.

Immediately upon opening the tool, we are greeted with the following user interface. We have the option to select among four workspaces.

- Import Model - Import an existing model in AML or XLSX format
- Build PPR Model - Build a new model from scratch
- Explore Sample - Demonstrates a pre-defined example
- Help and Docs - Information and documentation

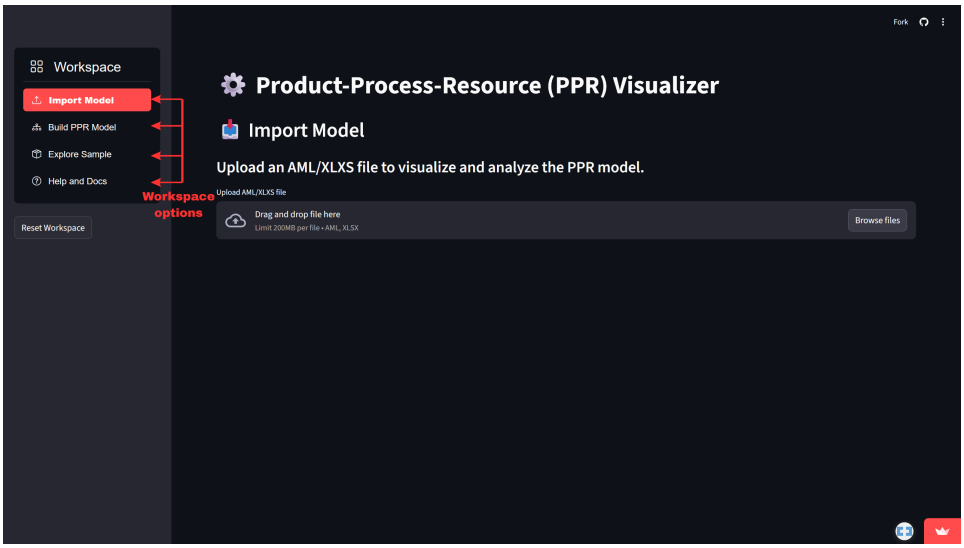


Figure 5.1: Initial User Interface

5.1 Import Model

The default workspace that we are greeted by is *Import Model*. Here we can upload an AML or XLSX file containing the PPR Model.

5.1.1 File input

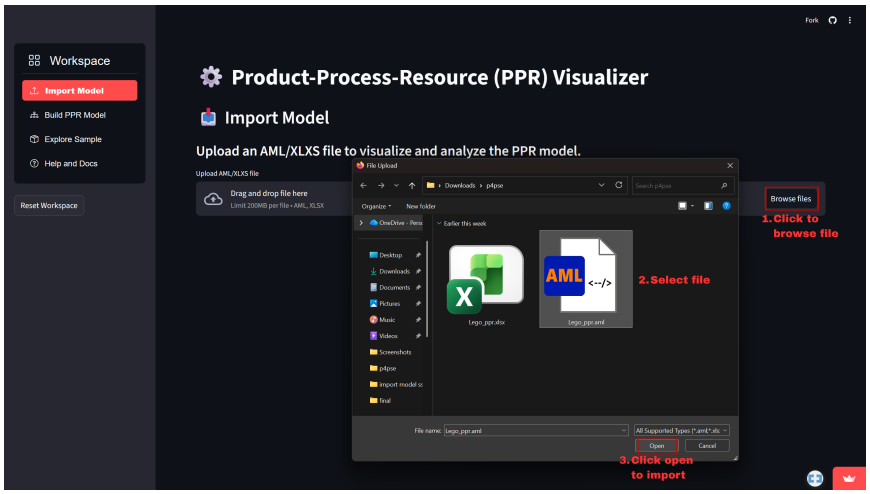


Figure 5.2: Steps for file input

5.1.2 User Interface

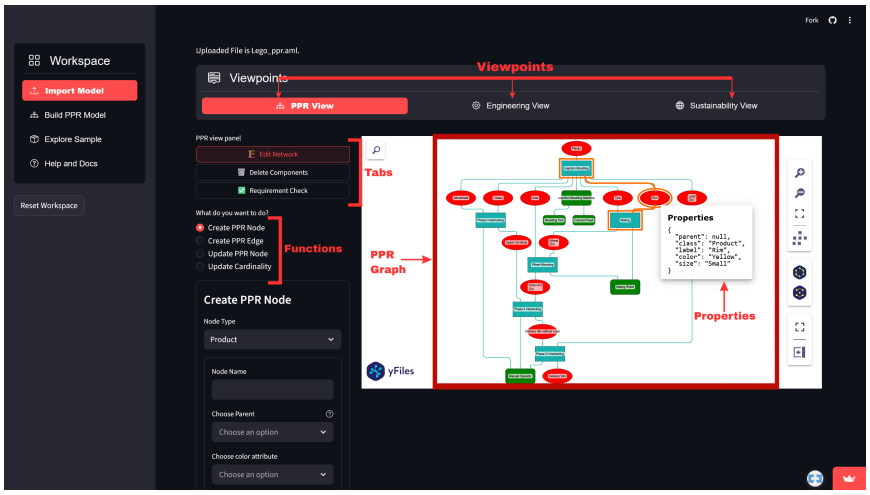


Figure 5.3: Import Model - User Interface

5.1.3 PPR View

Node Creation

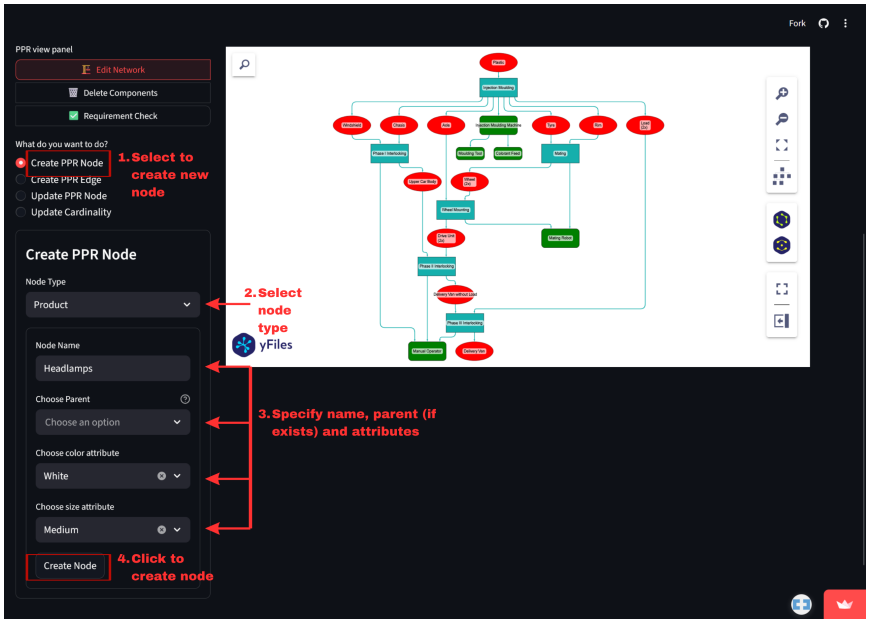


Figure 5.4: PPR View - Steps for creating new node

Edge Creation

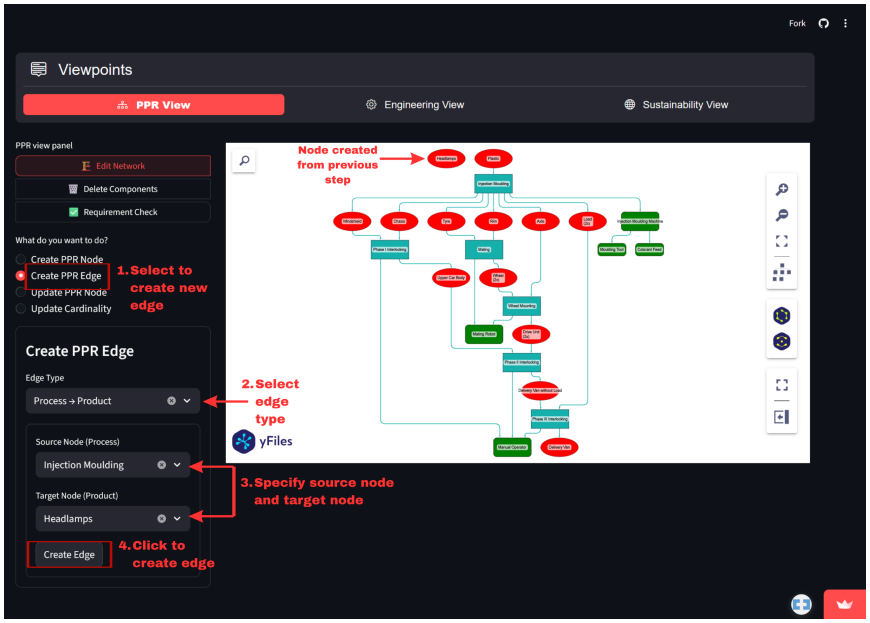


Figure 5.5: PPR View - Steps for creating new edge

Node Update

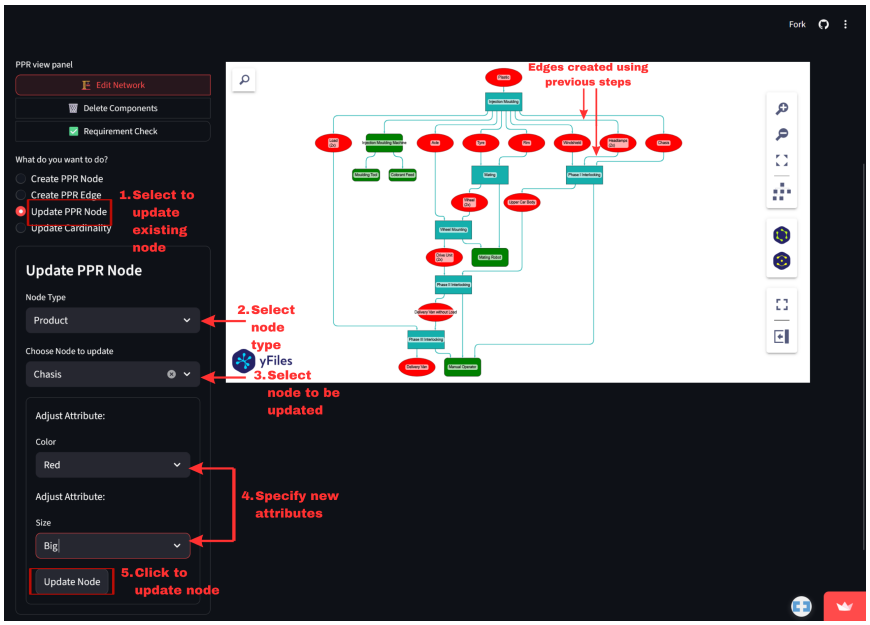


Figure 5.6: PPR View - Steps for updating node

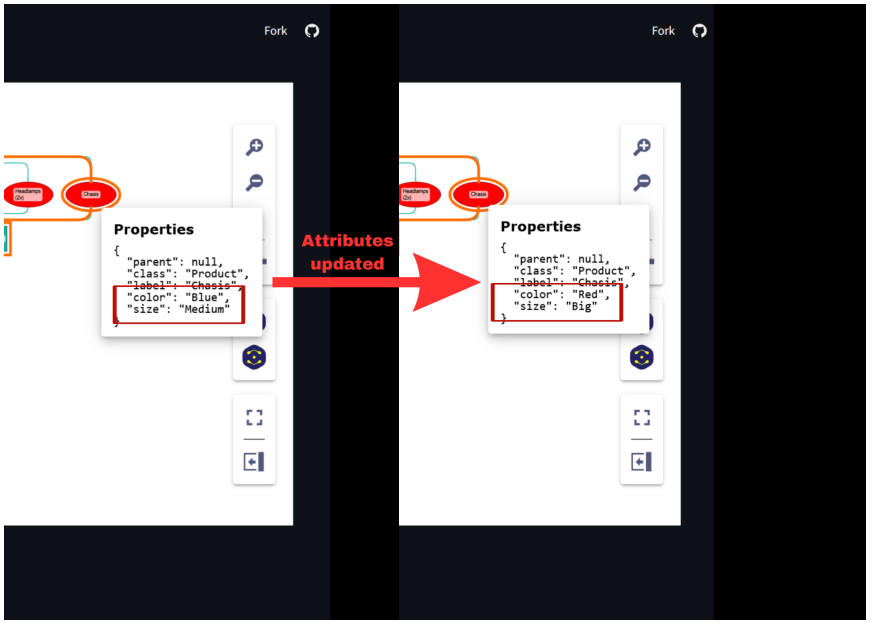


Figure 5.7: PPR View - Updated node

Cardinality Update

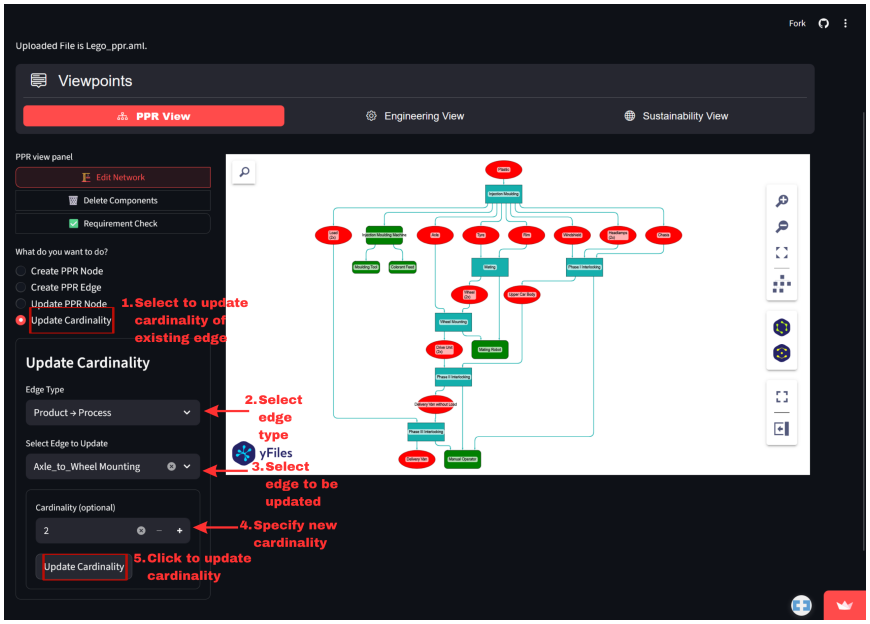


Figure 5.8: PPR View - Steps for updating cardinality

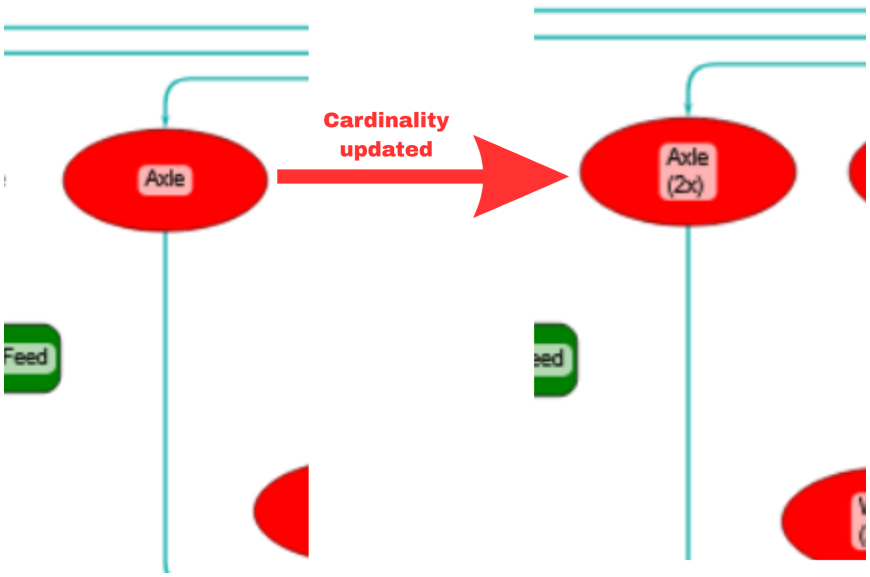


Figure 5.9: PPR View - Updated cardinality

Node Deletion

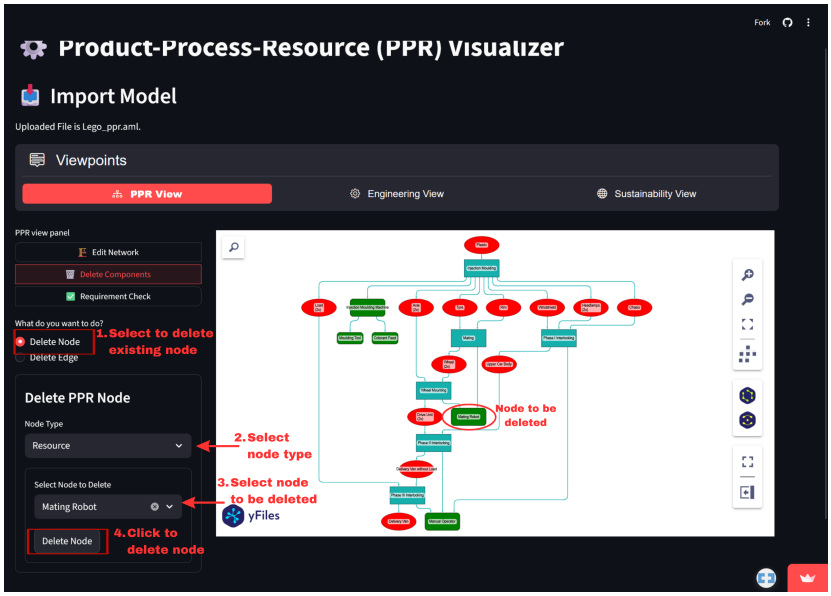


Figure 5.10: PPR View - Steps for deleting existing node

Edge Deletion

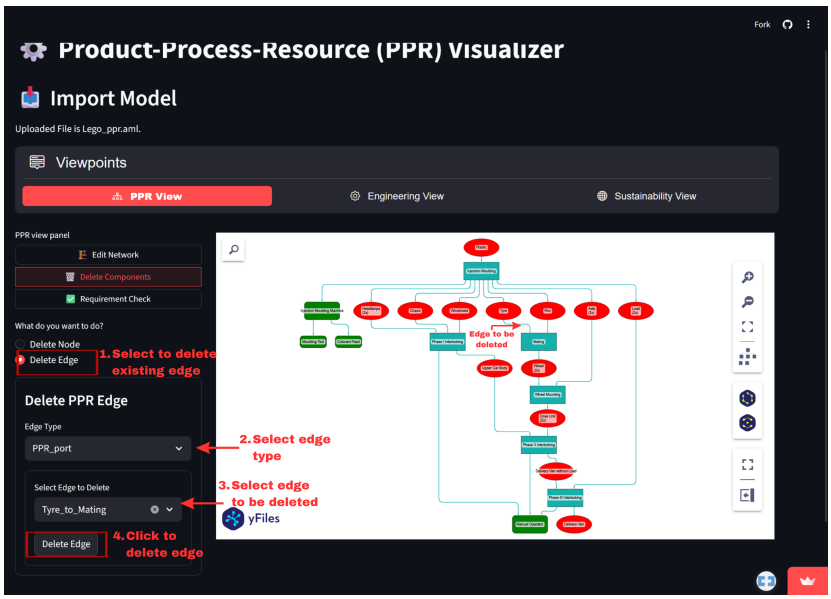


Figure 5.11: PPR View - Steps for deleting existing edge

Requirement Check

This tab lets us identify similarly structured elements and provides functions that can perform pre-defined requirement checks.

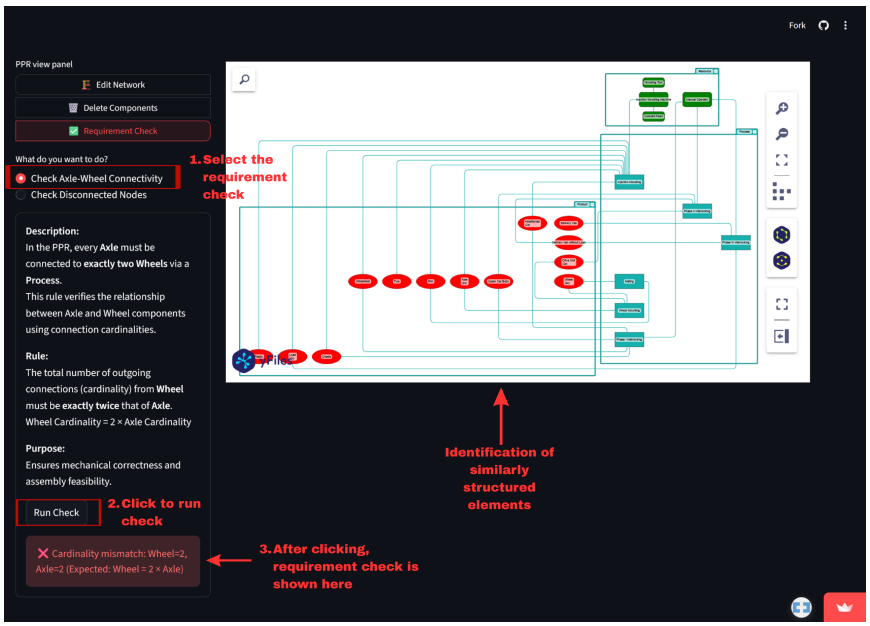


Figure 5.12: PPR View - Steps for performing requirement check

5.1.4 Engineering View

In *Engineering View*, we can model a new network using the existing nodes from the PPR View graph.
The following functions are available in the *Edit Network* tab.

Node Addition

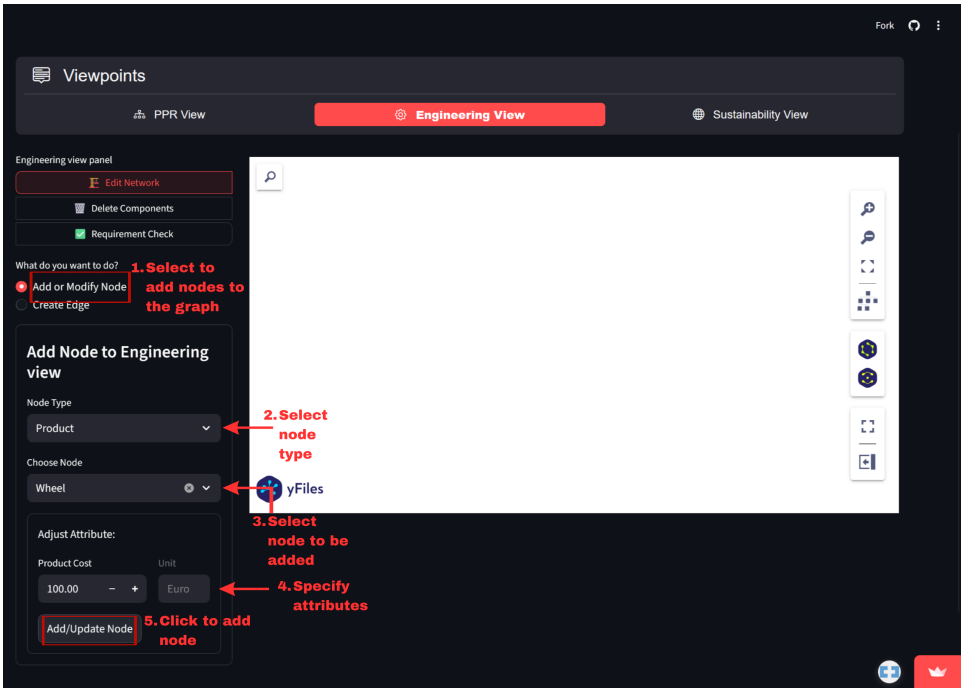


Figure 5.13: Engineering View - Steps for adding node

Edge Creation

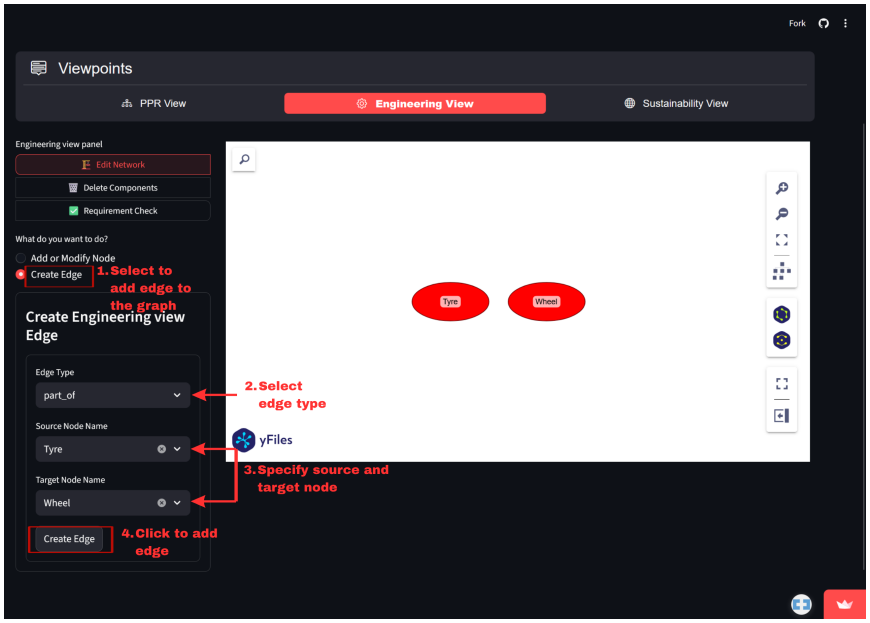


Figure 5.14: Engineering View - Steps for creating new edge

Node Deletion

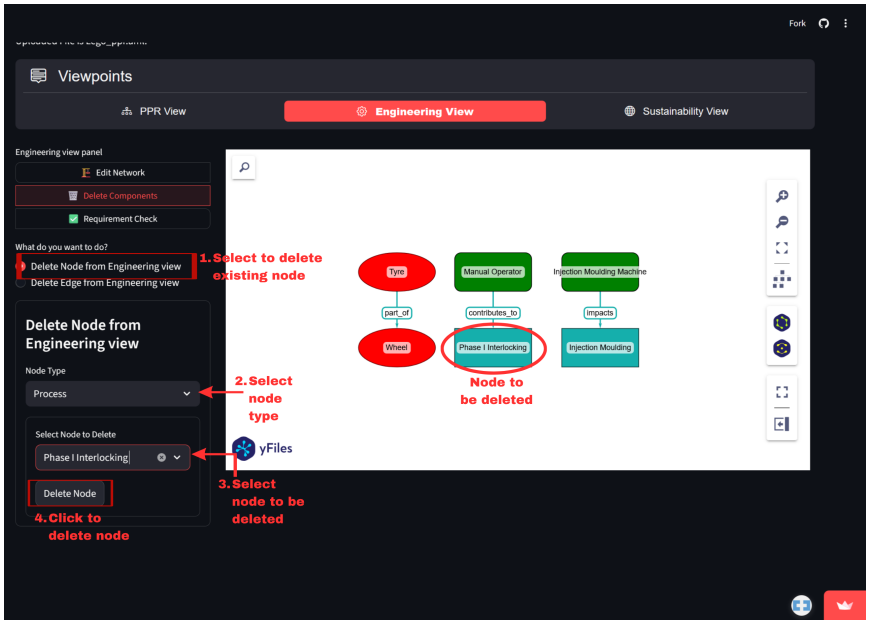


Figure 5.15: Engineering View - Steps for deleting existing node

Edge Deletion

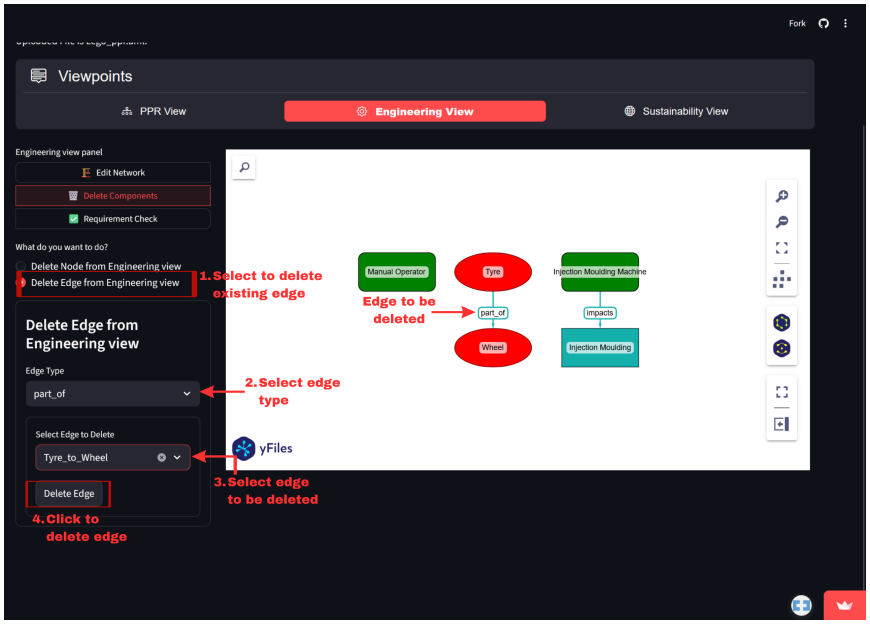


Figure 5.16: Engineering View - Steps for deleting existing edge

Requirement Check

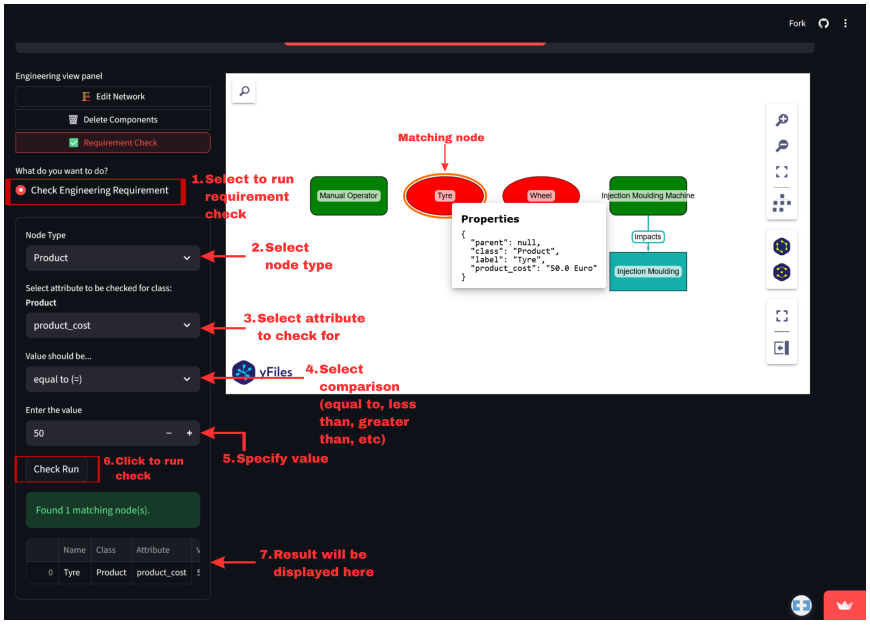


Figure 5.17: Engineering View - Steps for performing requirement check

5.1.5 Sustainability View

Similarly to *Engineering View*, in *Sustainability View*, we can model a new network using the existing nodes from the PPR View graph. The steps are identical to *Engineering View* and the same methodology can be followed for the different functions.

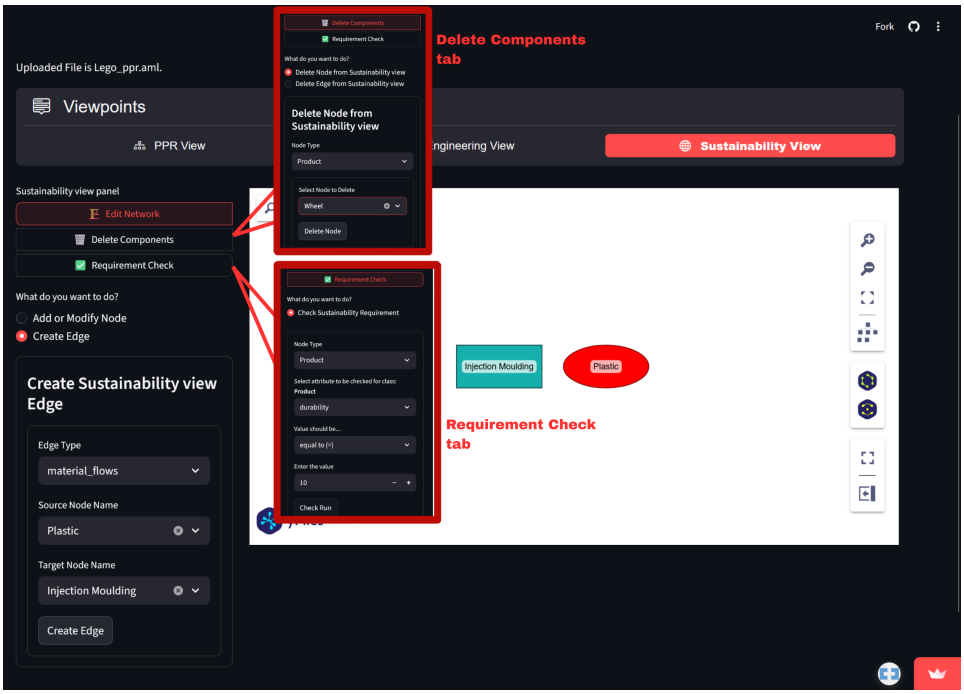


Figure 5.18: Sustainability View - User Interface

5.2 Build PPR Model

If the *Import Model* workspace is already active and we want to switch to the *Build PPR* workspace, a warning will appear. We can reset the workspace to start building a new model from scratch. The following steps illustrate how.

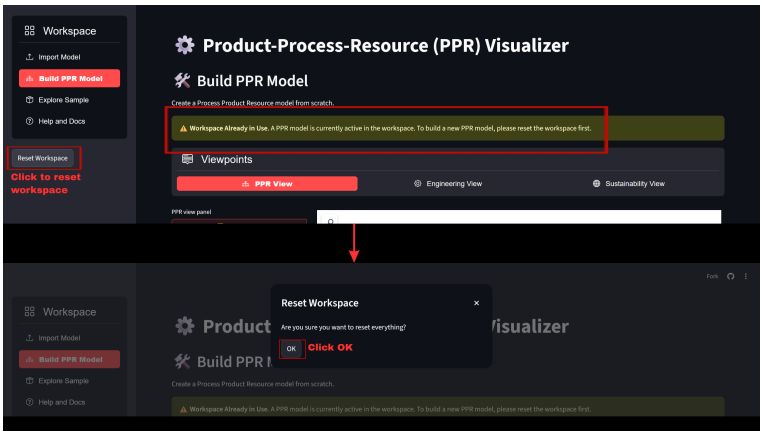


Figure 5.19: Steps for switching from *Import Model* to *Build PPR Model*

The following user interface will appear in the *Build PPR Model* workspace. We can follow the same methodology as we did for *Import Model* workspace for building a network and performing requirement checks for the different viewpoints, with the only difference being that we are starting anew rather than having an existing model file.

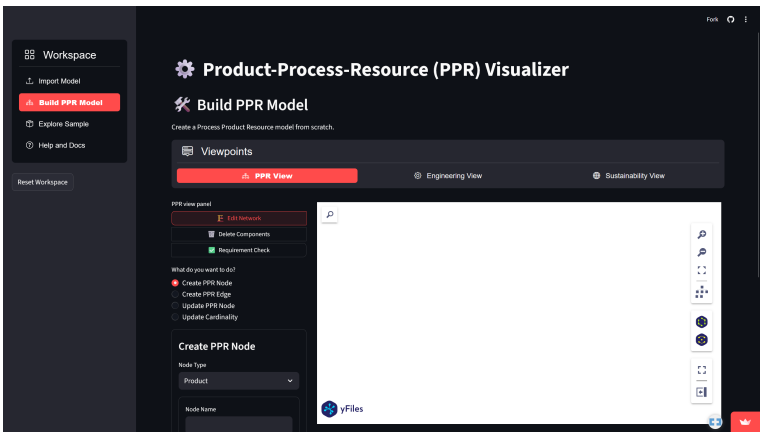


Figure 5.20: Build PPR Model - User Interface

5.3 Explore Sample

Here, we provide a predefined example to understand the structure of the model.

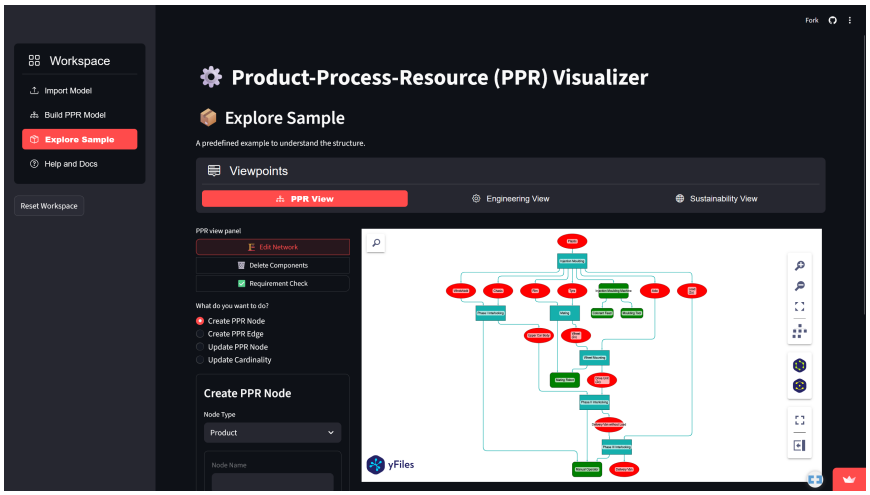


Figure 5.21: Explore Sample - User Interface

5.4 Help and Docs

Here, we provide information about the PPR format and conformity alongside the documentation.

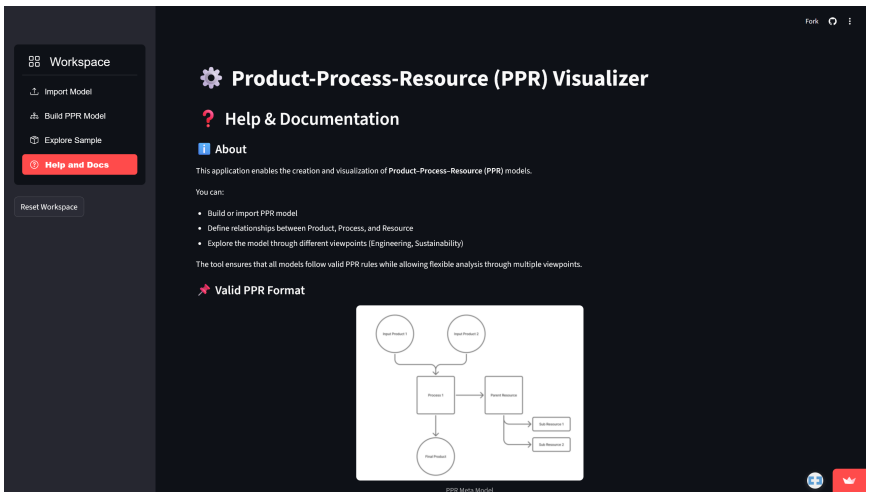


Figure 5.22: Help and Docs - User Interface